



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Primena kodova za ispravljanje grešaka u satelitskim komunikacijama

Seminarski rad

Studijski program: Elektrotehnika I Računarstvo

Modul: Računarstvo I informatika

Student:

Mateja Nešić, br. ind. 18796

Mentor:

Prof. Miloš Radmanović

Univerzitet u Nišu
Elektronski Fakultet

**PRIMENA KODOVA ZA ISPRAVLJANJE GREŠAKA U SATELITSKIM
KOMUNIKACIJAMA**

ERROR-CORRECTING CODES IN SATELLITE COMMUNICATIONS

Seminarski rad
Studijski program: Elektrotehnika I računarstvo
Modul: Računarstvo I informatika

Student: Mateja Nešić ,18796
Mentor: Prof. Miloš Radmanović

Datum prijave rada: 5.06.2026
Datum predaje rada: 26.06.2026
Datum odbrane rada: 29.06.2026

Komisija za ocenu i odbranu:

1. Prof. Miloš Radmanović, Predsednik Komisije

2. Prof. Nenad Petrović, Član

3. <Prof. / Doc dr Ime Prezime>, Član

SAŽETAK

U ovom seminarskom radu projektovan je i realizovan softverski simulacioni model digitalnog komunikacionog kanala u prisustvu aditivnog belog Gausovog šuma (AWGN), sa fokusom na primenu u satelitskim sistemima. Cilj rada je uporedna analiza performansi prenosa podataka bez zaštite, sa Hammingovim (7,4) kodom i sa kodom sa ponavljanjem (Repetition (3,1) kod) pri upotrebi BPSK modulacije. Evaluacija sistema izvršena je kroz analizu stope greške po bitu (BER), efikasnog propusnog opsega (Throughput), procesorskog vremena izvršavanja i stabilnosti sistema pri skaliranju količine podataka. Rezultati simulacije potvrđuju fundamentalni kompromis (*trade-off*) u telekomunikacijama između robusnosti sistema na šum, spektralne efikasnosti i hardverske kompleksnosti, dajući uvid u kriterijume za izbor optimalne tehnike kodiranja u realnim uslovima satelitskog prenosa.

ABSTRACT

This paper presents the design and implementation of a software simulation model for a digital communication channel in the presence of additive white Gaussian noise (AWGN), focusing on its application in satellite systems. The primary objective is a comparative performance analysis of data transmission without error correction, with Hamming (7,4) code, and with Repetition (3,1) code using BPSK modulation. The evaluation of the system was conducted by analyzing the Bit Error Rate (BER), effective throughput, processing execution time, and system stability under data scaling. The simulation results confirm the fundamental trade-off in telecommunications between noise robustness, spectral efficiency, and hardware complexity, providing insights into the criteria for selecting the optimal coding technique in realistic satellite transmission environments.

SADRŽAJ

1. UVOD.....	6
2. TEORIJSKA POZADINA.....	7
2.1. Koncept diskretnog komunikacionog kanala.....	7
2.2. Binarna fazna modulacija (BPSK).....	7
2.3. Model aditivnog belog Gausovog šuma (AWGN).....	7
2.4. Prijemnik i kriterijum odlučivanja (Demodulacija).....	8
2.5. Metrike za evaluaciju performansi sistema.....	8
3. TEORIJA USKO VEZANA ZA PROJEKAT.....	10
3.1. Hammingov (7,4) kod.....	10
3.2. Repetition (3,1) kod (Kod sa ponavljanjem).....	11
4. OPIS IZGLEDA I IMPLEMENTACIJE APLIKACIJE U PROJEKTU.....	12
4.1 Struktura izvornog koda i opis funkcija.....	12
4.2 Tok izvršavanja simulacije (simulaciona petlja).....	12
4.3 Korisnički izgled i uputstvo za pokretanje aplikacije.....	13
Uputstvo za pokretanje aplikacije:.....	13
5. EKSPERIMENTALNI REZULTATI.....	14
5.1 Test 1: Analiza stope greške po bitu (BER) u funkciji od SNR-a.....	14
5.2 Test 2: Analiza efikasnog propusnog opsega (Throughput).....	15
5.3 Test 3: Analiza efikasnosti i brzine koda (Coding Rate).....	15
5.4 Test 4: Analiza vremenske kompleksnosti u funkciji obima podataka.....	16
6. ZAKLJUČAK.....	18
6.1 Analiza performansi, prednosti i mana implementiranih metoda.....	18
LITERATURA.....	19
PRILOG– Izvorni kod implementacije.....	20

1. UVOD

Savremeni satelitski komunikacioni sistemi predstavljaju ključnu infrastrukturu za globalni prenos podataka, televizijskog i radio signala, kao i za potrebe navigacije i meteoroloških osmatranja. S obzirom na to da satelitski linkovi pokrivaju ogromne udaljenosti kroz kosmički prostor i Zemljinu atmosferu, signal na svom putu od predajnika do prijemnika trpi uticaj različitih nelinearnih izobličenja, atmosferskih slabljenja i različitih izvora šuma. Glavni izazov u ovakvim sistemima jeste očuvanje integriteta prenesenih informacija u uslovima izuzetno niskog odnosa signal-šum (SNR).

U realnim scenarijima, slanje podataka bez ikakve zaštite pri niskim vrednostima SNR-a dovodi do neprihvatljivo visoke stope greške po bitu (BER), čineći primljene podatke neupotrebljivim. Da bi se ovaj problem prevazišao, primenjuju se tehnike kontrole grešaka unapred, odnosno kodovi za ispravljanje grešaka (Error Correction Codes – ECC). Ove tehnike podrazumevaju kontrolisano uvođenje redundantnosti (zaštitnih bitova) u informacionu sekvencu na predajnoj strani, što prijemniku omogućava da detektuje i samostalno ispravi nastale greške bez potrebe za ponovnim slanjem paketa.

Cilj ovog seminarskog rada je projektovanje i realizacija softverskog modela satelitske komunikacije koji demonstrira primenu i performanse različitih ECC tehnika. Kroz rad su implementirana i analizirana dva pristupa: Hammingov (7,4) kod, kao predstavnik linearnih blok kodova sa umerenom efikasnošću, i kod sa ponavljanjem (Repetition code), kao primer ekstremno robusnog ali spektralno neefikasnog koda. Performanse sistema su evaluirane u prisustvu aditivnog belog Gausovog šuma (AWGN) kroz analizu stope greške (BER), propusnog opsega korisnih podataka (Throughput), kao i hardverske kompleksnosti izražene kroz vreme izvršavanja algoritama na prijemnoj strani.

2. TEORIJSKA POZADINA

U ovom poglavlju analiziraju se fundamentalni koncepti diskretne obrade signala i telekomunikacija koji predstavljaju osnovu za realizaciju simulacionog modela u ovom projektu. Prenos informacija kroz savremene komunikacione sisteme podrazumeva transformaciju digitalnih podataka u formu prilagođenu fizičkom medijumu, kao i matematičko modelovanje svih nesavršenosti kanala koje degradiraju kvalitet tog prenosa.

2.1. Koncept diskretnog komunikacionog kanala

Komunikacioni kanal u diskretnom vremenu posmatra se kao sistem koji prima niz simbola sa predajnika, vrši njihovu transmisiju kroz medijum i isporučuje ih prijemniku uz određeni nivo izobličenja. U sistemima digitalne obrade signala, primarni cilj je minimizacija uticaja šuma i smetnji uz maksimalno iskorišćenje raspoloživog frekvencijskog opsega i snage predajnika.

U opštem slučaju, performanse kanala direktno zavise od tri ključna faktora:

- Tipa primenjene modulacije (fazna, amplitudska ili frekvencijska),
- Karakteristika samog fizičkog medijuma (slabljenje, refleksija, termički šum),
- Prisustva sistema za kontrolu i ispravljanje grešaka (ECC).

2.2. Binarna fazna modulacija (BPSK)

Da bi se binarni niz bita preneo kroz kontinualni kanal, neophodno je izvršiti modulaciju. U satelitskim komunikacijama, gde su energetske resursi satelita kritično ograničeni, binarna fazna modulacija (Binary Phase Shift Keying – BPSK) predstavlja standard zbog svoje visoke otpornosti na šum i optimalnog iskorišćenja snage.

BPSK funkcioniše tako što mapira binarne simbole (bitove) d_i iz skupa $\{0, 1\}$ u kontinualne talasne oblike promenom faze nosećeg signala za 180° . U diskretnom domenu, ovaj proces se pojednostavljuje na nivo naponskih nivoa gde se vrši linearno mapiranje:

$$s_i = 2d_i - 1$$

Na osnovu ove relacije, bit 1 se transformiše u amplitudu $+1$ V, dok se bit 0 transformiše u amplitudu -1 V. Geometrijski posmatrano u signalnom prostoru, ovi simboli predstavljaju dve antipodalne tačke na realnoj osi, čime se postiže maksimalno moguće rastojanje između simbola za zadatu snagu signala.

2.3. Model aditivnog belog Gausovog šuma (AWGN)

Tokom prolaska kroz satelitski downlink, na modulisani signal utiče veliki broj nezavisnih slučajnih faktora (termički šum u poluprovodnicima prijemnika, kosmičko zračenje, šum atmosfere). Prema Centralnoj graničnoj teoremi, ukupni efekat svih ovih pojedinačnih izvora smetnji teži normalnoj (Gausovoj) raspodeli. Zbog toga se za analizu performansi koristi model kanala sa aditivnim belim Gausovim šumom (AWGN – Additive White Gaussian Noise).

Matematički model primljenog signala r_i u diskretnom vremenu opisuje se relacijom:

$$r_i = s_i + n_i$$

gde je s_i modulirani predajni simbol, a n_i slučajna promenljiva koja modeluje šum. Karakteristike ovog šuma su:

- **Aditivan:** Šum se direktno sabira sa korisnim signalom.
- **Beo:** Posедуje konstantnu spektralnu gustinu snage kroz sve frekvencije.
- **Gausov:** Gustina raspodele verovatnoće amplitude šuma definisana je normalnom raspodelom sa srednjom vrednošću nula i varijansom σ^2 :

$$p(n) = \frac{1}{\sigma \sqrt{2\pi}} e^{\frac{-n^2}{2\sigma^2}}$$

Intenzitet šuma u kanalu meri se preko odnosa signal-šum (SNR – Signal-to-Noise Ratio), koji predstavlja odnos snage korisnog signala i snage šuma, a u telekomunikacijama se po pravilu izražava u decibelima (dB):

$$SNR_{dB} = 10 \log_{10} \left(\frac{P_{signal}}{P_{\text{šum}}} \right)$$

2.4. Prijemnik i kriterijum odlučivanja (Demodulacija)

Na prijemnoj strani, demodulator ima zadatak da na osnovu kontaminiranog primljenog signala r_i donese odluku o tome koji je bit stvarno poslat. S obzirom na to da su BPSK simboli simetrično postavljeni u odnosu na koordinatni početak (+1 V i -1 V), optimalni prijemnik postavlja prag odlučivanja tačno na vrednost 0 V.

Algoritam odlučivanja (demodulacije) funkcioniše po sledećem principu:

$$d_i = \begin{cases} 1 & \text{za } r_i > 0 \\ 0 & \text{za } r_i \leq 0 \end{cases}$$

Ukoliko je intenzitet šuma toliki da amplituda signala pređe prag odlučivanja (npr. poslat je bit 1 odnosno +1 V, ali je šum naneo vrednost -1.2 V, pa je primljeno -0.2 V), prijemnik donosi pogrešnu odluku i upisuje bit 0.

2.5. Metrike za evaluaciju performansi sistema

Za objektivnu procenu kvaliteta komunikacionog sistema i efikasnosti koda koriste se dve ključne, međusobno zavisne metrike:

- **BER (Bit Error Rate):** Stopa greške po bitu predstavlja osnovnu metriku pouzdanosti. Definše se kao odnos broja pogrešno detektovanih bita (N_{errors}) i ukupnog broja poslatih bita (N_{total}) tokom simulacije:

$$BER = \frac{N_{errors}}{N_{total}}$$

- **Throughput (Propusni opseg korisnih podataka):** Predstavlja meru spektralne efikasnosti sistema. Dok BER meri tačnost, throughput meri realnu brzinu prenosa korisnih informacija. Kada se primenjuju kodovi za ispravljanje grešaka, throughput zavisi i od stope greške, ali i od samog kodnog koeficijenta (R), jer zaštitni bitovi troše resurse kanala. Izračunava se kao:

$$Throughput = R \cdot (1 - BER)$$

gde je R kodni koeficijent (R=1 za prenos bez zaštite).

3. TEORIJA USKO VEZANA ZA PROJEKAT

U ovom poglavlju detaljno se analiziraju matematički modeli, struktura i algoritmi dva primenjena koda za kontrolu i ispravljanje grešaka unapred (Forward Error Correction – FEC) koji su realizovani u okviru softverskog modela: Hammingov (7,4) kod i kod sa ponavljanjem (Repetition (3,1) kod).

3.1. Hammingov (7,4) kod

Hammingov kod spada u klasu linearnih blok kodova i predstavlja jedan od najpoznatijih i najefikasnijih klasičnih kodova za sisteme sa umerenim nivoom šuma. Označava se opštom sintaksom (n, k) , gde je dužina kodne reči $n = 7$ bita, a broj korisnih informacionih bita $k = 4$. Razlika $n - k = 3$ predstavlja broj zaštitnih bita (parity bits) koji se proračunavaju nad informacionim bitovima i šalju zajedno sa njima kroz komunikacioni kanal.

Proces kodiranja se vrši na predajnoj strani. Korisna poruka se deli na blokove dužine 4 bita, koji se mogu predstaviti kao vektor u :

$$u = u_1 \quad u_2 \quad u_3 \quad u_4$$

Ovaj informacioni vektor se množi sa generatorskom matricom G , dimenzija 4×7 , po modulu 2 (ekvivalentno ekskluzivnoj disjunktiji – XOR operaciji):

$$c = (u \cdot G)$$

Generatorska matrica G , koja je direktno implementirana u Python kodu ovog projekta, ima sledeću fiksnu strukturu:

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Prve četiri kolone ove matrice čine jediničnu matricu, što znači da je kod sistematski – prva 4 bita kodne reči c su identična originalnim informacionim bitovima, dok su preostala 3 bita zaštitna.

Na prijemnoj strani, nakon demodulacije, dobija se primljeni vektor c koji može sadržati greške usled šuma. Detekcija greške se vrši računanjem vektora sindroma s , dimenzija 3×1 , množenjem matrice kontrole parnosti H (dimenzija 3×7) sa transponovanim primljenim vektorom, po modulu 2:

$$s = (H \cdot c^T)$$

Matrica kontrole parnosti H definisana u projektu iznosi:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Interpretacija sindroma s vrši se na sledeći način:

- Ako je vektor sindroma jednak nuli (s sve nule), prijemnik zaključuje da je reč preneti bez greške.
- Ako je sindrom različit od nule, algoritam vrši poređenje vektora s sa kolonama matrice H. Kolona matrice H koja je identična vektoru sindroma direktno ukazuje na indeks (poziciju) bita u kome se desila greška. Prijemnik zatim vrši invertovanje tog bita (XOR sa 1) i uspešno ispravlja grešku.

Hammingov kod ima minimalno Hammingovo rastojanje $d_{\min} = 3$, što mu daje teorijsku sposobnost da ispravi tačno jednu grešku (Single Error Correction) ili da detektuje maksimalno dve greške u bloku od 7 bita.

3.2. Repetition (3,1) kod (Kod sa ponavljanjem)

Kod sa ponavljanjem predstavlja najjednostavniju tehniku kodiranja sa kontrolom grešaka. U ovom projektu realizovana je varijanta (3,1), gde se svaki pojedinačni informacioni bit ($k = 1$) replikuje (ponavlja) tri puta ($n = 3$) pre nego što se pošalje u kanal sa šumom.

Pravilo kodiranja je izuzetno prosto:

- Ako je informacioni bit $d = 1$, šalje se kodna reč $c = [1, 1, 1]$
- Ako je informacioni bit $d = 0$, šalje se kodna reč $c = [0, 0, 0]$

Na prijemnoj strani se vrši dekodovanje po principu logike glasanja većine (Majority Logic Decoding). Algoritam sabira vrednosti sva tri primljena bita u bloku. Ukoliko je zbir veći ili jednak 2, donosi se odluka da je poslat bit 1. U suprotnom, odluka je bit 0.

Matematički, pravilo odlučivanja za primljeni blok od tri bita r_1, r_2, r_3 izgleda ovako:

$$\hat{d} = \begin{cases} 1 & \text{za } \sum_{i=1}^3 r_i \geq 2 \\ 0 & \text{za } \sum_{i=1}^3 r_i < 2 \end{cases}$$

Ovaj kod pruža izuzetno veliku robusnost pri ekstremno visokom šumu (niskom SNR-u) jer je potrebno da šum pokvari čak dva od tri bita u istom bloku da bi prijemnik pogrešio. Međutim, cena ove robusnosti je ogromna spektralna neefikasnost – kodni koeficijent iznosi svega $R = 1/3$ (svega 33.3% poslatih bita su stvarni podaci), što drastično degradira brzinu prenosa.

4. OPIS IZGLEDA I IMPLEMENTACIJE APLIKACIJE U PROJEKTU

U ovom poglavlju opisan je softverski paket razvijen u programskom jeziku Python (uz korišćenje biblioteka numpy, matplotlib i time), koji u potpunosti simulira satelitski komunikacioni kanal sa i bez primene ECC tehnika. Kod je struktuiran modularno, kroz jasno definisane funkcije za kodiranje, modulaciju, simulaciju šuma, demodulaciju i dekodovanje.

4.1 Struktura izvornog koda i opis funkcija

Implementacija je podeljena na logičke blokove koji odgovaraju blok-šemi jednog savremenog telekomunikacionog sistema. U nastavku je dat detaljan opis ključnih funkcija:

- `encode_hamming(bits)`: Prima niz informacionih bita i vrši njihovo preoblikovanje u matricu dimenzija $N \times 4$ (blokovi od 4 bita), a zatim vrši matricno množenje sa generatorskom matricom G po modulu 2. Rezultat je matrica kodovanih blokova dužine 7 bita.
- `decode_hamming(codeword)`: Realizuje sindromsko dekodovanje. Funkcija prvo računa vektor sindroma množenjem sa matricom H . Ukoliko detektuje grešku, algoritam prolazi kroz kolone matrice H , pronalazi pokvareni bit, vrši njegovu korekciju (XOR operacija) i na kraju izdvaja prva 4 korisna bita.
- `encode_repetition(bits)`: Koristi ugrađenu `numpy.repeat` funkciju kako bi svaki ulazni bit replikovala tri puta, generišući tako kodovanu sekvencu za kod sa ponavljanjem.
- `decode_repetition(bits)`: Deli primljeni niz na blokove od po 3 bita, sabira ih i primenjuje logiku većinskog odlučivanja (proverava da li je zbir veći ili jednak 2).
- `bpsk_mod(bits)` i `bpsk_demod(symbols)`: Vrš preslikavanje bita u naponske nivoe $+1\text{ V}$ / -1 V i obrnuto, na osnovu praga odlučivanja na nuli.
- `awgn(signal, snr_db)`: Simulira uticaj šuma u kanalu. Na osnovu zadate vrednosti SNR u decibelima, funkcija izračunava snagu signala, generiše belo Gausovo odstupanje preko `np.random.randn` i sabira ga sa korisnim signalom.

4.2 Tok izvršavanja simulacije (simulaciona petlja)

Glavna funkcija `simulate(snr_db_range, n_bits)` pokreće kompletan test za opseg SNR vrednosti od 0 do 10 dB sa korakom 1 dB. Za svaku tačku generiše se nasumični niz od 4000 bita, koji se zatim šalje kroz tri paralelne grane:

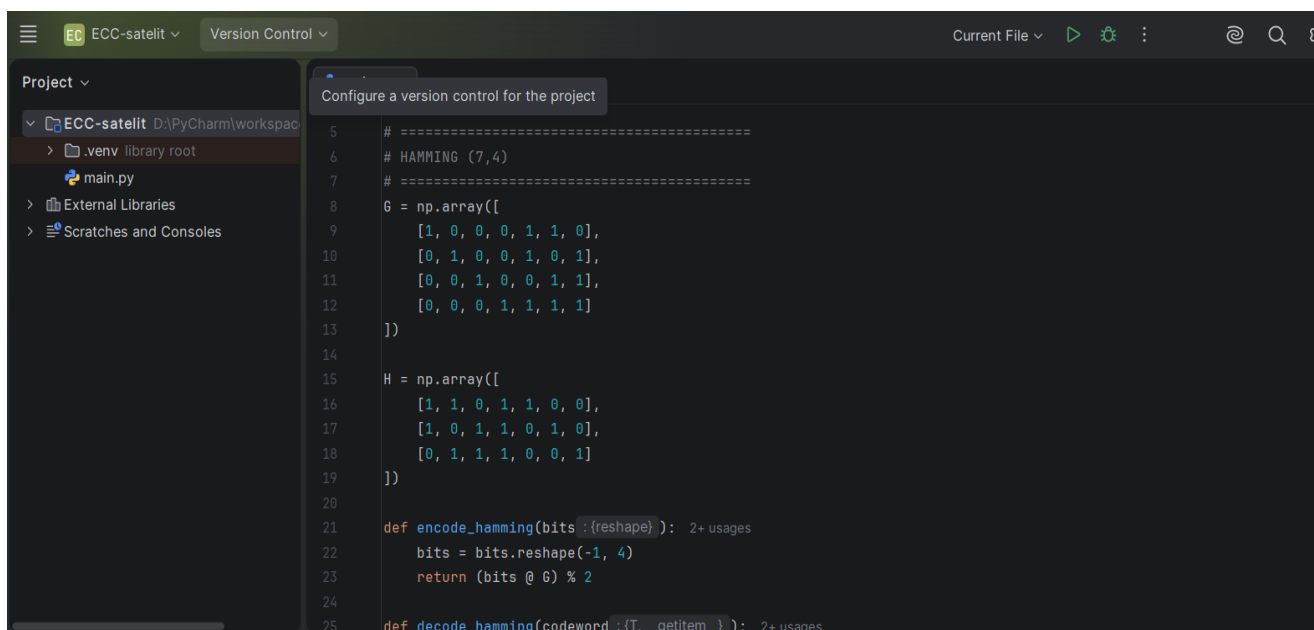
- Grana bez zaštite (No ECC)
- Grana sa Hammingovim (7,4) kodom
- Grana sa kodom sa ponavljanjem (Repetition)

Nakon završetka simulacije kanala, računa se konačna stopa greške po bitu i throughput za svaku granu, što omogućava generisanje uporednih grafika performansi.

4.3 Korisnički izgled i uputstvo za pokretanje aplikacije

Aplikacija je realizovana kao izvršna Python skripta koja se pokreće u razvojnom okruženju (poput PyCharm-a ili VS Code-a). Program ne zahteva dodatni unos parametara od strane korisnika, već automatski izvršava simulaciju sa predefinisanim parametrima visokog prioriteta (4000 bita za analizu BER i Throughput metrika, kao i do 32000 bita za testiranje memorijskog i vremenskog opterećenja sistema).

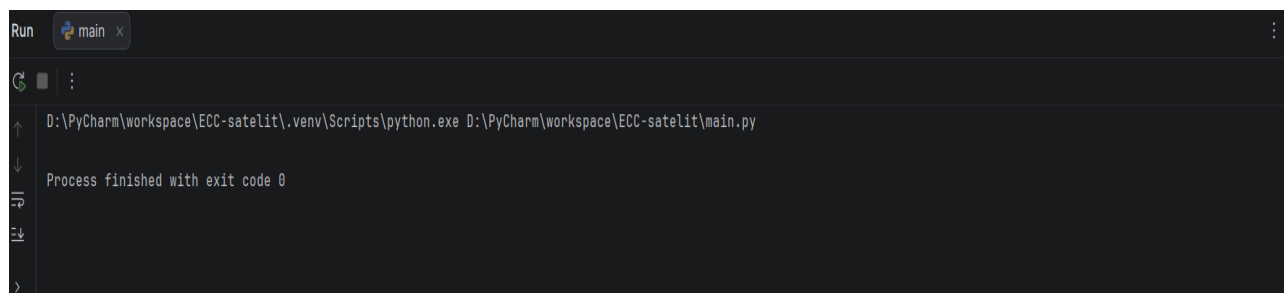
Nakon uspešnog izvršavanja algoritama, aplikacija automatski generiše i prikazuje četiri grafička prozora koji omogućavaju vizuelnu analizu dobijenih rezultata.



Slika 4.1 – Izgled razvojnog okruženja PyCharm sa otvorenim izvornim kodom aplikacije.

Uputstvo za pokretanje aplikacije:

- Uveriti se da je instaliran Python (verzija 3.8 ili novija).
- Instalirati neophodne biblioteke pokretanjem komande u terminalu:
pip install numpy matplotlib
- Otvoriti fajl sa izvornim kodom u izabranom razvojnom okruženju.
- Pokrenuti skriptu (prečica Shift + F10 u PyCharm-u ili opcija Run u meniju).
- Aplikacija će redom prikazati četiri grafika. Zatvaranjem jednog grafika automatski se otvara sledeći.



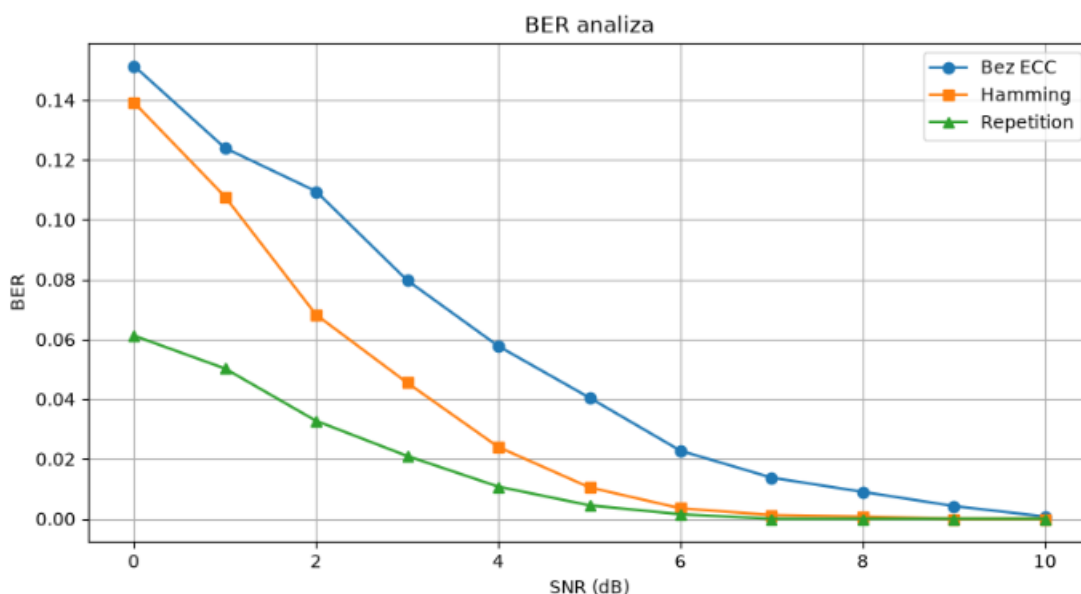
Slika 4.2: Uspešno izvršavanje skripte u terminalu bez sintaksnih ili logičkih grešaka.

5. EKSPERIMENTALNI REZULTATI

U ovom poglavlju prikazani su rezultati simulacije prenosa podataka kroz AWGN kanal i izvršena je uporedna analiza performansi sistema u tri scenarija: bez primene zaštite (No ECC), sa Hammingovim (7,4) kodom i sa Repetition (3,1) kodom. Evaluacija je izvršena kroz četiri različita testa kako bi se obuhvatile i telekomunikacione metrike i hardverska kompleksnost algoritama.

5.1 Test 1: Analiza stope greške po bitu (BER) u funkciji od SNR-a

Prvi eksperiment obuhvata analizu stope greške po bitu (Bit Error Rate – BER) za vrednosti odnosa signal-šum (SNR) u opsegu od 0 dB do 10 dB sa korakom od 1 dB. Veličina testiranog bloka podataka iznosi 4000 bita.



Slika 5.1 – Uporedni prikaz stope greške po bitu (BER) u funkciji od SNR-a.

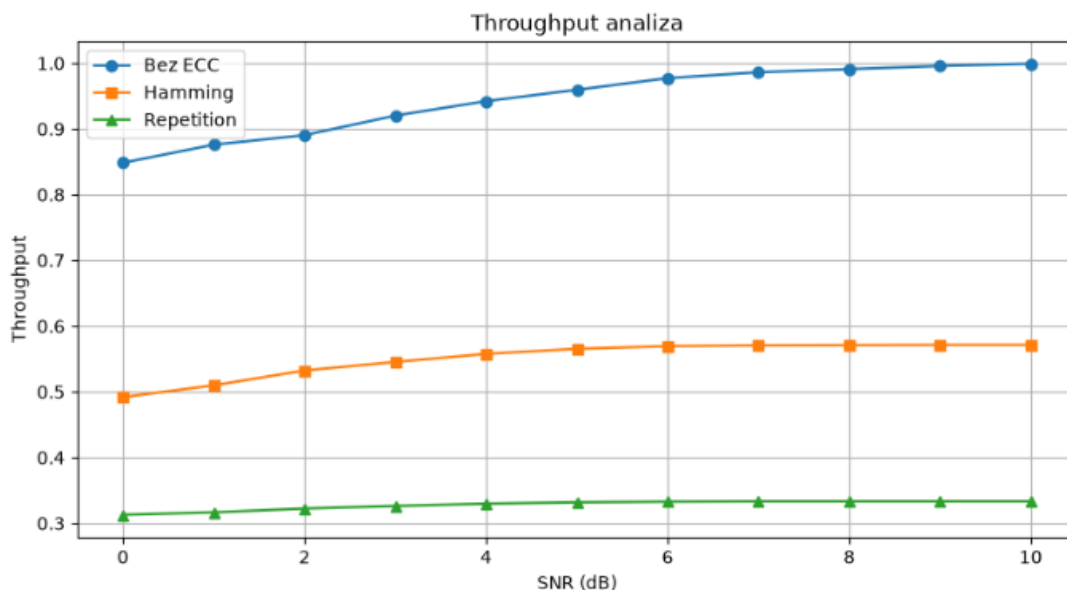
Analiza rezultata

Na osnovu generisanog grafika na Slici 5.1 jasno se uočava prednost primene kodova za ispravljanje grešaka. Prenos bez zaštite (plava linija) ima najlošije performanse kroz ceo opseg.

- Repetition (3,1) kod (zeleno linija) pokazuje najbolje performanse na niskim vrednostima SNR-a (0–4 dB), jer logika većinskog odlučivanja uspešno eliminiše uticaj jakog šuma.
- Hammingov (7,4) kod (narandžasta linija) na veoma niskom SNR-u ima slabije rezultate, jer se u blokovima od 7 bita često javlja više od jedne greške, što prevazilazi njegov kapacitet korekcije. Međutim, kako SNR raste iznad 5 dB, Hammingov kod pokazuje nagli pad krive (tzv. waterfall efekat) i postaje znatno efikasniji, obezbeđujući visoku pouzdanost uz manji broj redundantnih bita u odnosu na kod sa ponavljanjem.

5.2 Test 2: Analiza efikasnog propusnog opsega (Throughput)

Drugi eksperiment analizira throughput (udeo uspešno prenesenih korisnih informacija) u zavisnosti od nivoa šuma u kanalu, za isti opseg SNR vrednosti.



Slika 5.2 – Uporedni prikaz efikasnog propusnog opsega (Throughput) u funkciji od SNR-a.

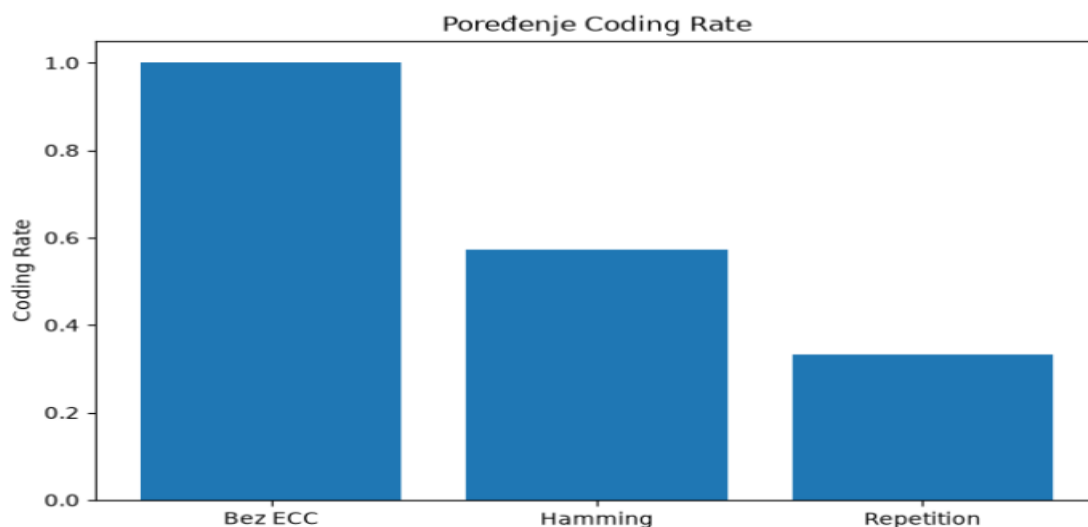
Analiza rezultata:

Grafik na Slici 5.2 jasno pokazuje kompromis između robusnosti i efikasnosti kodova.

- Sistem bez zaštite na visokom SNR-u dostiže maksimalni teoretski throughput od 1.0, jer ne koristi redundantne bitove, ali na niskom SNR-u vrednost naglo opada zbog velikog broja grešaka.
- Hammingov (7,4) kod ima maksimalni mogući throughput ograničen na $R = 4/7 \approx 0.57$. Ovu vrednost stabilno postiže već od oko 5 dB, što pokazuje da uspešno održava stabilnu komunikaciju i efikasno ispravlja greške.
- Repetition (3,1) kod je uprkos dobroj otpornosti na šum spektralno neefikasan. Njegov throughput je ograničen na $R = 1/3 \approx 0.33$, što znači da se čak i u idealnim uslovima dve trećine kapaciteta kanala troše na redundantne kopije istih podataka.

5.3 Test 3: Analiza efikasnosti i brzine koda (Coding Rate)

Treći eksperiment analizira parametar efikasnosti koda (*Coding Rate*), odnosno odnos između broja informacionih (korisnih) bitova i ukupnog broja bitova koji se prenose kroz komunikacioni kanal za sve tri posmatrane metode.



Slika 5.3 – Poređenje Coding Rate parametra za različite tehnike zaštite.

Analiza rezultata:

Sa histograma prikazanog na Slici 5.4 jasno se uočava kompromis između nivoa zaštite i spektralne efikasnosti sistema.

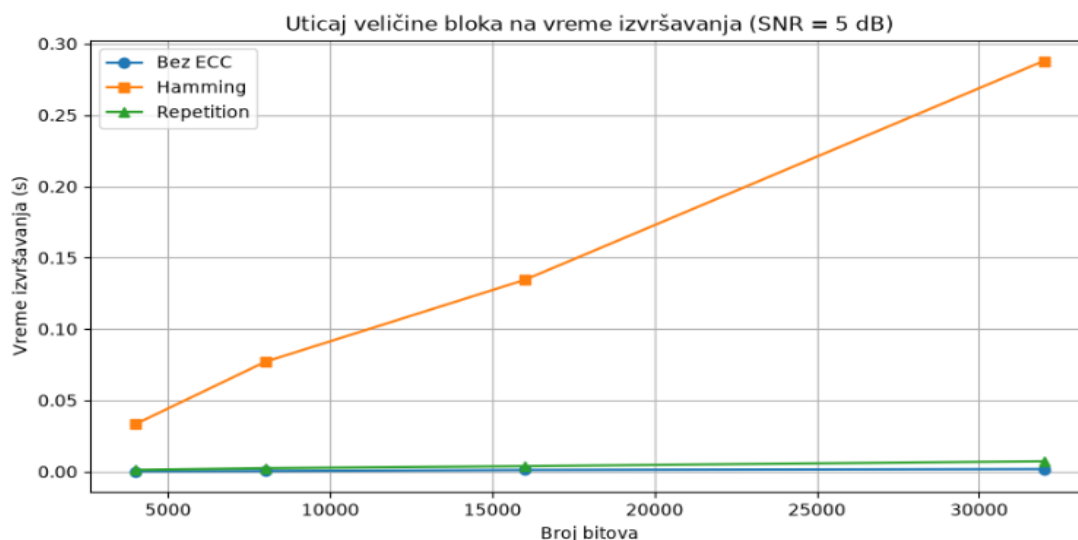
Prenos bez zaštite (Bez ECC) ostvaruje maksimalnu efikasnost sa coding rate vrednošću 1.0. To znači da sistem ne uvodi nikakvu redundantnost i da svi preneti bitovi predstavljaju isključivo korisne informacije. Međutim, ovakav sistem je izrazito osetljiv na prisustvo šuma u kanalu, što je potvrđeno visokom stopom greške (BER) u prethodnim testovima.

Hammingov (7,4) kod ostvaruje efikasnost od približno 0.57 (odnos 4/7). Ovaj rezultat pokazuje da oko 57% ukupnog protoka čine korisni podaci, dok je preostalih 43% rezervisano za zaštitne, odnosno paritetne bitove. Sa stanovišta projektovanja komunikacionih sistema, Hammingov kod predstavlja veoma dobro kompromisno rešenje, jer omogućava korekciju pojedinačnih grešaka uz umereno smanjenje efektivne brzine prenosa.

Repetition (3,1) kod pokazuje najnižu efikasnost, koja iznosi približno 0.33 (odnos 1/3). Da bi se obezbedila visoka otpornost sistema na šum, svaki bit se prenosi tri puta, što znači da se čak dve trećine kapaciteta kanala troše na redundantne informacije. Iako ovaj pristup pruža visoku robusnost u uslovima izraženog šuma, njegova spektralna efikasnost je veoma niska, što ga čini nepovoljnim za prenos velikih količina podataka.

5.4 Test 4: Analiza vremenske kompleksnosti u funkciji obima podataka

Četvrti eksperiment analizira ponašanje sistema i računarsko opterećenje pri promeni ukupne količine podataka (veličine ulaznog fajla) u opsegu od 4000 do 32000 bita, uz fiksnu vrednost šuma na kanalu od 5 dB.



Slika 5.4 –Uticaj skaliranja veličine ulaznih podataka na vreme izvršavanja algoritma.

Analiza rezultata:

Sa grafikona prikazanog na Slici 5.4 jasno se vidi da ukupno vreme izvršavanja raste približno linearno sa povećanjem broja ulaznih bitova kod sve tri analizirane metode. Povećanjem obima podataka sa 4000 na 32000 bita dolazi do proporcionalnog rasta vremena obrade, što potvrđuje očekivano ponašanje simulacionog sistema pri većim opterećenjima.

Linearni rast vremena izvršavanja je i teorijski i praktično očekivan, jer povećanje broja ulaznih bitova direktno utiče na broj blokova koje sistem mora da generiše, moduliše, propusti kroz AWGN kanal, demoduliše i na kraju dekoduje.

Prenos bez zaštite (Bez ECC) pokazuje najmanje vreme izvršavanja kroz ceo opseg testiranih veličina. Razlog za to je jednostavna obrada na prijemnoj strani, gde se dekodovanje svodi isključivo na demodulaciju i poređenje sa pragom odlučivanja.

Repetition (3,1) kod pokazuje umereno povećanje vremena izvršavanja. Iako ovaj pristup uvodi trostruko veći broj bitova za prenos, proces dekodovanja je relativno jednostavan i zasniva se na sabiranju blokova i logici većinskog odlučivanja.

Hammingov (7,4) kod ima najveće vreme izvršavanja među svim analiziranim metodama. Glavni razlog leži u složenijem procesu dekodovanja, posebno tokom sindromske analize. Za svaki primljeni blok algoritam mora da izvrši matrice operacije po modulu 2, a zatim i dodatnu pretragu radi identifikacije i korekcije eventualne greške.

Dobijeni rezultati potvrđuju kompromis između performansi i procesorske složenosti: jednostavniji sistemi nude bržu obradu ali slabiju otpornost na šum, dok naprednije ECC tehnike obezbeđuju veću pouzdanost uz povećane hardverske i procesorske zahteve.

6. ZAKLJUČAK

Kroz realizaciju ovog seminarskog rada uspešno je projektovan i analiziran simulacioni model satelitske komunikacije u prisustvu aditivnog belog Gausovog šuma (AWGN), uz primenu BPSK modulacije i dve različite tehnike kontrole grešaka (ECC). Na osnovu teorijske analize i eksperimentalnih rezultata dobijenih kroz četiri sprovedena testa, izvedeni su ključni zaključci o ponašanju sistema, efikasnosti algoritama i kompromisima koji prate savremene telekomunikacione sisteme.

6.1 Analiza performansi, prednosti i mana implementiranih metoda

Eksperimentalni rezultati jasno ukazuju na to da svaka testirana metoda prenosa poseduje specifične prednosti i mane, te da izbor optimalnog rešenja direktno zavisi od uslova u kanalu i raspoloživih hardverskih resursa.

Prenos podataka bez primene zaštite (No ECC) odlikuje se maksimalnom brzinom i potpunim iskorišćenjem kapaciteta kanala, što potvrđuje efikasni propusni opseg (Throughput) sa konstantnom vrednošću od 1.0 u idealnim uslovima. Takođe, ovaj pristup zahteva minimalnu procesorsku kompleksnost, jer na prijemnoj strani nema dekodovanja. Međutim, njegova glavna mana je izuzetna osetljivost na šum. Pri niskim vrednostima SNR-a, stopa greške (BER) raste do neprihvatljivih nivoa, čineći prenos bez zaštite potpuno neupotrebljivim u satelitskim komunikacijama gde su smetnje konstantno prisutne.

Sa druge strane, kod sa ponavljanjem (Repetition (3,1) kod) pokazao je visoku otpornost u uslovima jakog šuma. Zahvaljujući logici većinskog odlučivanja na prijemniku, ovaj kod uspešno eliminiše greške na niskim vrednostima SNR-a i obezbeđuje stabilnu vezu tamo gde drugi sistemi otkazuju. Ipak, cena ove robusnosti je njegova najveća mana — izrazita spektralna neefikasnost. Pošto se svaki bit šalje tri puta, maksimalni propusni opseg korisnih podataka ograničen je na 0.33, što znači da se čak i u idealnim uslovima dve trećine kapaciteta kanala troše na redundantnost.

Hammingov (7,4) kod predstavlja izbalansirano i inženjerski efikasno rešenje koje uspešno premošćuje razliku između prethodna dva pristupa. Njegova glavna prednost ogleda se u većoj spektralnoj efikasnosti u poređenju sa kodom sa ponavljanjem, jer ostvaruje stabilan throughput od 0.57. Iako pri ekstremno niskom SNR-u dolazi do degradacije performansi zbog pojave višestrukih grešaka u jednom bloku, Hammingov kod sa porastom odnosa signal-šum pokazuje nagli pad stope greške i uspostavlja stabilnu i pouzdanu komunikaciju. Glavna mana ovog koda je veća hardverska kompleksnost, jer sindromsko dekodovanje i matične operacije po modulu 2 zahtevaju više procesorskog vremena na prijemnoj strani.

LITERATURA

- [1] Nenad Petrović, *I deo – Error correction kodovi (implementacija)*, Elektronski fakultet, Niš, 2021.
- [2] David Salomon, *Data Compression: The Complete Reference*, Springer, 5th Edition, 2010.
- [3] Khalid Sayood, *Introduction to Data Compression*, Morgan Kaufmann, 5th Edition, 2017.
- [4] Rafael C. Gonzalez, Richard E. Woods, *Digital Image Processing*, Pearson Education, 4th Edition, 2018.
- [5] John G. Proakis, Masoud Salehi, *Digital Communications*, McGraw-Hill, 5th Edition, 2008.
- [6] Simon Haykin, *Communication Systems*, John Wiley & Sons, 4th Edition, 2001.
- [7] R. W. Hamming, "Error detecting and error correcting codes," *The Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, Apr. 1950.
- [8] Claude E. Shannon, "A Mathematical Theory of Communication," *The Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, Oct. 1948.
- [9] NumPy Developers, "NumPy Reference Guide - Array manipulation routines," NumPy.org. [Onlajn]. <https://numpy.org/doc/>.
- [10] MathWorks, "Binary Phase Shift Keying (BPSK) Modulation in AWGN Channel," MATLAB & Simulink Documentation. [Onlajn].

PRILOG– Izvorni kod implementacije

```
import numpy as np
import matplotlib.pyplot as plt
import time

# =====
# HAMMING (7,4)
# =====
G = np.array([
    [1, 0, 0, 0, 1, 1, 0],
    [0, 1, 0, 0, 1, 0, 1],
    [0, 0, 1, 0, 0, 1, 1],
    [0, 0, 0, 1, 1, 1, 1]
])

H = np.array([
    [1, 1, 0, 1, 1, 0, 0],
    [1, 0, 1, 1, 0, 1, 0],
    [0, 1, 1, 1, 0, 0, 1]
])

def encode_hamming(bits):
    bits = bits.reshape(-1, 4)
    return (bits @ G) % 2

def decode_hamming(codeword):
    syndrome = (H @ codeword.T) % 2
    syndrome = syndrome.T

    corrected = []
    for i, s in enumerate(syndrome):
        error_pos = None
        for j in range(7):
            if np.array_equal(H[:, j], s):
                error_pos = j
                break

        cw = codeword[i].copy()
        if error_pos is not None:
            cw[error_pos] ^= 1
        corrected.append(cw[:4])

    return np.array(corrected).reshape(-1)

# =====
# REPETITION CODE (3,1)
# =====
def encode_repetition(bits):
    return np.repeat(bits, 3)

def decode_repetition(bits):
    bits = bits.reshape(-1, 3)
    return (np.sum(bits, axis=1) >= 2).astype(int)

# =====
# BPSK
```

```

# =====
def bpsk_mod(bits):
    return 2 * bits - 1

def bpsk_demod(symbols):
    return (symbols > 0).astype(int)

# =====
# AWGN
# =====
def awgn(signal, snr_db):
    snr = 10 ** (snr_db / 10)
    power = np.mean(signal ** 2)
    noise_power = power / snr
    noise = np.sqrt(noise_power) * np.random.randn(len(signal))
    return signal + noise

# =====
# BER TEST
# =====
def simulate(snr_db_range, n_bits=4000):
    ber_no = []
    ber_hamming = []
    ber_rep = []

    throughput_no = []
    throughput_hamming = []
    throughput_rep = []

    for snr_db in snr_db_range:
        bits = np.random.randint(0, 2, n_bits)

        # =====
        # NO ECC
        # =====
        tx = bpsk_mod(bits)
        rx = awgn(tx, snr_db)
        rx_bits = bpsk_demod(rx)

        ber = np.mean(bits != rx_bits)
        ber_no.append(ber)
        throughput_no.append(1 * (1 - ber))

        # =====
        # HAMMING
        # =====
        bits_h = bits[:((len(bits)//4)*4)]

        coded = encode_hamming(bits_h)
        tx_h = bpsk_mod(coded.flatten())
        rx_h = awgn(tx_h, snr_db)
        rx_bits_h = bpsk_demod(rx_h)

        decoded = decode_hamming(rx_bits_h.reshape(-1, 7))
        ber = np.mean(bits_h != decoded[:len(bits_h)])
        ber_hamming.append(ber)
        throughput_hamming.append((4/7)*(1-ber))

        # =====
        # REPETITION

```

```

# =====
rep = encode_repetition(bits)
tx_r = bpsk_mod(rep)
rx_r = awgn(tx_r, snr_db)
rx_bits_r = bpsk_demod(rx_r)

decoded_r = decode_repetition(rx_bits_r)
ber = np.mean(bits != decoded_r)
ber_rep.append(ber)
throughput_rep.append((1/3)*(1-ber))

return (ber_no, ber_hamming, ber_rep,
        throughput_no, throughput_hamming, throughput_rep)

# =====
# BLOCK SIZE TEST
# =====
def block_size_test():
    sizes = [4000, 8000, 16000, 32000]

    times_no = []
    times_hamming = []
    times_rep = []

    for size in sizes:
        bits = np.random.randint(0, 2, size)

        # =====
        # NO ECC
        # =====
        start = time.time()
        tx = bpsk_mod(bits)
        rx = awgn(tx, 5)
        rx_bits = bpsk_demod(rx)
        elapsed = time.time() - start
        times_no.append(elapsed)

        # =====
        # HAMMING
        # =====
        bits_h = bits[:((len(bits)//4)*4)]

        start = time.time()
        coded = encode_hamming(bits_h)
        tx = bpsk_mod(coded.flatten())
        rx = awgn(tx, 5)
        rx_bits = bpsk_demod(rx)
        decode_hamming(rx_bits.reshape(-1, 7))
        elapsed = time.time() - start
        times_hamming.append(elapsed)

        # =====
        # REPETITION
        # =====
        start = time.time()
        rep = encode_repetition(bits)
        tx = bpsk_mod(rep)
        rx = awgn(tx, 5)
        rx_bits = bpsk_demod(rx)
        decode_repetition(rx_bits)

```

```

    elapsed = time.time() - start
    times_rep.append(elapsed)

    return sizes, times_no, times_hamming, times_rep
# =====
# RUN
# =====
snr_db_range = np.arange(0, 11, 1)

(ber_no, ber_hamming, ber_rep,
 throughput_no, throughput_hamming, throughput_rep) = simulate(snr_db_range)

sizes, times_no, times_hamming, times_rep = block_size_test()

# =====
# GRAF 1 - BER
# =====
plt.figure(figsize=(10, 5))
plt.plot(snr_db_range, ber_no, 'o-', label="Bez ECC")
plt.plot(snr_db_range, ber_hamming, 's-', label="Hamming")
plt.plot(snr_db_range, ber_rep, '^-', label="Repetition")
plt.xlabel("SNR (dB)")
plt.ylabel("BER")
plt.title("BER analiza")
plt.grid()
plt.legend()
plt.show()

# =====
# GRAF 2 - Throughput
# =====
plt.figure(figsize=(10, 5))
plt.plot(snr_db_range, throughput_no, 'o-', label="Bez ECC")
plt.plot(snr_db_range, throughput_hamming, 's-', label="Hamming")
plt.plot(snr_db_range, throughput_rep, '^-', label="Repetition")
plt.xlabel("SNR (dB)")
plt.ylabel("Throughput")
plt.title("Throughput analiza")
plt.grid()
plt.legend()
plt.show()

# =====
# GRAF 3 - Block Size
# =====
plt.figure(figsize=(10, 5))
plt.plot(sizes, times_no, 'o-', label="Bez ECC")
plt.plot(sizes, times_hamming, 's-', label="Hamming")
plt.plot(sizes, times_rep, '^-', label="Repetition")

plt.xlabel("Broj bitova")
plt.ylabel("Vreme izvršavanja (s)")
plt.title("Uticaj veličine bloka na vreme izvršavanja (SNR = 5 dB)")
plt.grid()
plt.legend()
plt.show()

# =====
# GRAF 4 - Coding Rate
# =====

```

```
methods = ["Bez ECC", "Hamming", "Repetition"]  
rates = [1, 4/7, 1/3]  
  
plt.figure(figsize=(8, 5))  
plt.bar(methods, rates)  
plt.ylabel("Coding Rate")  
plt.title("Poređenje Coding Rate")  
plt.show()
```